Build a modern, production-quality desktop streaming application using **Electron + React + TypeScript + Node.js**.

The application should stream a selected application window or desktop using **WebRTC** with very low latency (target under 200ms).

Requirements

Architecture

- Electron desktop application
- React + TypeScript frontend
- Node.js backend
- WebRTC for media streaming
- WebSocket signaling server
- Modular architecture
- Clean folder structure
- Production-ready code

Features

Broadcaster

- Select monitor
- Select application window
- Capture microphone
- Capture system audio (Windows)
- Preview stream
- Start Stream
- Stop Stream
- Generate shareable stream ID

Viewer

- Enter Stream ID
- Join stream
- Adaptive bitrate
- Auto reconnect
- Display connection quality
- Fullscreen support

Streaming

- WebRTC PeerConnection

- ICE candidate exchange
- STUN support
- TURN support
- Hardware encoding if available
- H264 preferred
- VP9 fallback
- Screen sharing
- Window sharing
- 60 FPS support
- Up to 1440p resolution

Networking

Implement signaling using WebSocket.

Messages:

- create-room
- join-room
- offer
- answer
- ice-candidate
- disconnect
- heartbeat

UI

Modern dark theme

Similar quality to:

- Discord
- OBS
- Steam

Include:

- animated transitions
- responsive layout
- beautiful cards
- status indicators
- latency indicator
- bitrate indicator
- FPS indicator

- viewer count

Performance

- Hardware acceleration
- Low CPU usage
- Efficient rendering
- Memory optimization
- Zero unnecessary rerenders

Security

- Room IDs
- Optional passwords
- DTLS encryption
- SRTP encryption
- Input validation

Extras

- Clipboard copy for room IDs
- QR code to join
- Chat
- Push-to-talk
- Screenshot button
- Recording option
- Stream statistics
- Automatic bitrate adaptation

Code Quality

- TypeScript everywhere
- ESLint
- Prettier
- Clean Architecture
- Reusable hooks
- Context API
- Proper error handling
- Loading states
- Unit-testable code

Folder Structure

Create a scalable folder structure for:

- Electron main process
- Electron preload
- React app
- Components
- Hooks
- Services
- WebRTC
- Signaling
- Utilities
- Types

Deliverables

Build the application step-by-step.
Start with project initialization, then signaling server, then WebRTC implementation, then Electron integration, then UI, then optimization.
Explain each major architectural decision before generating code